

Artificial Intelligence - Report Lab 4 & 5

Anh Khoi Pham - ITCSIU23018

May 5, 2026

1 Exercise 1

Problem: Implement the constraint satisfaction problem in the initialize function in the csp.py. How did you represent the Sudoku puzzle a CSP? What design options did you consider, and how did you decide on this implementation?

```
1  class csp:
2      # INITIALIZING THE CSP
3      def __init__(self, domain=digits, grid=""):
4          """
5          Unitlist consists of the 27 lists of
6              peers
7          Units is a dictionary consisting of the
8              keys and the corresponding lists of
9              peers
10         Peers is a dictionary consisting of the
11             81 keys and the corresponding set of
12             27 peers
13         Constraints denote the various all-
14             different constraints between the
15             variables
16         """
17         "***YOUR CODE HERE ***"
18         self.domain = domain
19         self.variables = squares
20         self.values = self.getDict(grid.strip())
21
22         # Unitlist: 9 rows, 9 columns, 9 boxes of
23             3x3
24         self.unitlist = (
25             [cross(rows, c) for c in cols] +
26             [cross(r, cols) for r in rows] +
27             [cross(rs, cs) for rs in ('ABC', 'DEF',
28                 'GHI') for cs in ('123', '456', '789')]
29         )
30
31         # Units: current key's rows, columns, and
32             box
33         self.units = dict((s, [u for u in self.
34             unitlist if s in u]) for s in self.
35             variables)
```

```
24
25     # Peers: all the squares that share a
      unit with the current key
26     # sum(self.units[s],[]) flattens the list
      of lists into a single list of peers
27     # set() removes duplicates
28     self.peers = dict((s, set(sum(self.units[
      s],[]))-set([s])) for s in self.
      variables)
29
30     self.constraints = []
31     for s in self.variables:
32         for peer in self.peers[s]:
33             self.constraints.append((s, peer)
34                                     )
35
36     def getDict(self, grid=""):
37         """
38         Getting the string as input and returning
39         the corresponding dictionary
40         """
41         i = 0
42         values = dict()
43         for cell in self.variables:
44             if grid[i] != '0':
45                 values[cell] = grid[i]
46             else:
47                 values[cell] = digits
48             i = i + 1
49         return values
```

First, we need to initialize four main components to construct the Sudoku problem as a CSP.

1. **variables:** This contains all 81 cells in the Sudoku grid, stored in `self.variables = squares`. Each cell is treated as one CSP variable.
2. **values:** This stores the domain of each variable. If a cell is already filled in the input grid, its domain is that single digit. Otherwise, its domain is `digits`, meaning it can still take any value from 1 to 9.
3. **unitlist and units:** `unitlist` stores the 27 units of Sudoku, including 9 rows, 9 columns, and 9 boxes. Then, `units` maps each cell to the

three units it belongs to, which makes it easy to identify the structural relationships of that cell.

4. **peers** and **constraints**: **peers** stores all neighboring cells that share a row, column, or box with a given cell. Based on these peers, **constraints** is created as a list of variable pairs that must satisfy the all-different condition, meaning two connected cells cannot take the same value.

This representation is effective because Sudoku naturally fits the CSP framework: each cell is a variable, each possible digit is a domain value, and the row, column, and box rules become constraints between related cells. I considered representing constraints separately for rows, columns, and boxes, but using **units** and **peers** is more compact and avoids repeated logic. It also works well with backtracking and forward checking, since the algorithm can quickly access the affected neighbors whenever a value is assigned.

2 Exercise 2

Problem: Implement Backtracking Search algorithm in the search.py.

```
1 def Backtracking_Search(csp):
2     return Recursive_Backtracking({}, csp)
3
4
5 def Recursive_Backtracking(assignment, csp):
6     if isComplete(assignment):
7         return assignment
8
9     var = Select_Unassigned_Variables(assignment,
10                                     csp)
11
12     for value in Order_Domain_Values(var,
13                                     assignment, csp):
14         if isConsistent(var, value, assignment,
15                         csp):
16             assignment[var] = value
17             inferences = forward_checking(csp,
18                                         assignment, var, value)
19             if inferences != "FAILURE":
20                 result = Recursive_Backtracking(
21                     assignment, csp)
22                 if result != "FAILURE":
```

```
18         return result
19     del assignment[var]
20     csp.values[var] = csp.domain
21     for neighbor in csp.peers[var]:
22         csp.values[neighbor] = csp.domain
23
24     return "FAILURE"
```

The backtracking algorithm solves the Sudoku puzzle by assigning values to empty cells step by step. First, `Select_Unassigned_Variables` chooses an unassigned cell. Next, `Order_Domain_Values` provides the possible values for that cell. For each candidate value, `isConsistent` checks whether the assignment violates any Sudoku constraint by comparing it with the cell's `peers`, which are the cells in the same row, column, or 3×3 box.

If a value is consistent, it is added to the current assignment, and the algorithm recursively continues with the next unassigned cell. If no valid value can be assigned, the algorithm backtracks by removing the previous assignment and trying another value. This process continues until all cells are assigned valid values, which means the puzzle has been solved.

3 Exercise 3

Problem: Implement AC-3 search algorithm.

```
1  def AC3_Search(csp):
2      """
3      AC3 search which calls the AC3 algorithm and
4      checks if the resulting CSP is solved
5      """
6      AC3(csp)
7
8      if all(len(csp.values[s]) == 1 for s in csp.
9             variables):
10         return csp.values
11     else:
12         return "FAILURE"
13
14 def AC3(csp):
15     """
```

```
15     AC3 algorithm which makes the CSP arc
      consistent
16     """
17     queue = [(Xi, Xj) for Xi in csp.variables for
               Xj in csp.peers[Xi]]
18
19     while queue:
20         Xi, Xj = queue.pop(0)
21         if Remove_Inconsistent_Values(csp, Xi, Xj
                                         ):
22             for Xk in csp.peers[Xi]:
23                 if Xk != Xj:
24                     queue.append((Xk, Xi))
25
26
27 def Remove_Inconsistent_Values(csp, Xi, Xj):
28     if len(csp.values[Xj]) == 1:
29         v = csp.values[Xj]
30         if v in csp.values[Xi]:
31             csp.values[Xi] = csp.values[Xi].
               replace(v, '')
32         return True
33     return False
```

The AC-3 algorithm solves the Sudoku puzzle by enforcing arc consistency on the CSP. It begins by creating a queue of arcs, where each arc represents a pair of related cells connected through Sudoku constraints. Then, it repeatedly removes one arc from the queue and checks whether the domain of one variable should be reduced according to the current domain of the other variable.

In this implementation, `Remove_Inconsistent_Values` removes a value from `Xi` when a peer `Xj` already has a single fixed value that conflicts with it. Whenever a domain is changed, all related arcs are added back to the queue so the effect can be propagated through the puzzle. If every variable is eventually reduced to a single value, the puzzle is solved. Therefore, AC-3 improves efficiency by narrowing domains early and eliminating inconsistent values before further search is needed.

4 Comparison

Table 1: Performance Comparison of Backtracking and AC-3 on the Euler Sudoku Dataset

Method	Solved	Total	Time (s)	Avg/Board (s)
Backtracking	50	50	24.6754	0.4935
AC-3	12	50	0.2065	0.0041

The experimental results show that Backtracking achieved a complete solution for all 50 Sudoku boards, while AC-3 solved only 12 boards. However, AC-3 required much less computation time because it only performs constraint propagation and does not continue with deeper search when propagation alone is insufficient. In contrast, Backtracking is significantly slower, but it is more complete because it explores alternative assignments until a valid solution is found. Therefore, AC-3 is faster but less reliable as a standalone solver, whereas Backtracking is slower but more effective for solving all puzzles in the dataset.